5

第5章 Thymeleaf と Web ページデザイン

章の目標

- Thymeleaf の基本構文について理解できる
- Thymeleaf から CSS や JavaScript を活用できる
- レイアウト機能について理解できる

Ｍａｒｋｄｏｗｎから生成した元位置再現版

5

第5章 Thymeleaf と Web ページデザイン

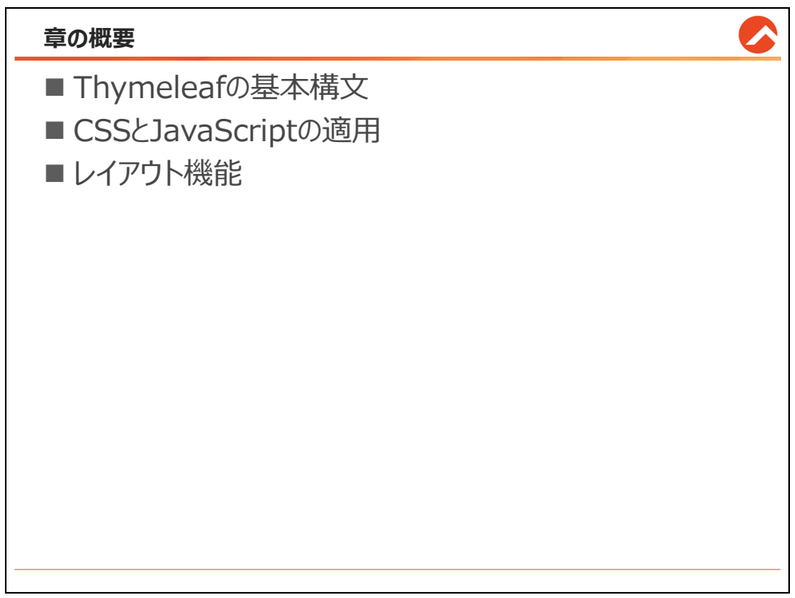
章の目標

- Thymeleaf の基本構文について理解できる
- Thymeleaf から CSS や JavaScript を活用できる
- レイアウト機能について理解できる

第 5 章 ThymeleafとWebページデザイン

165

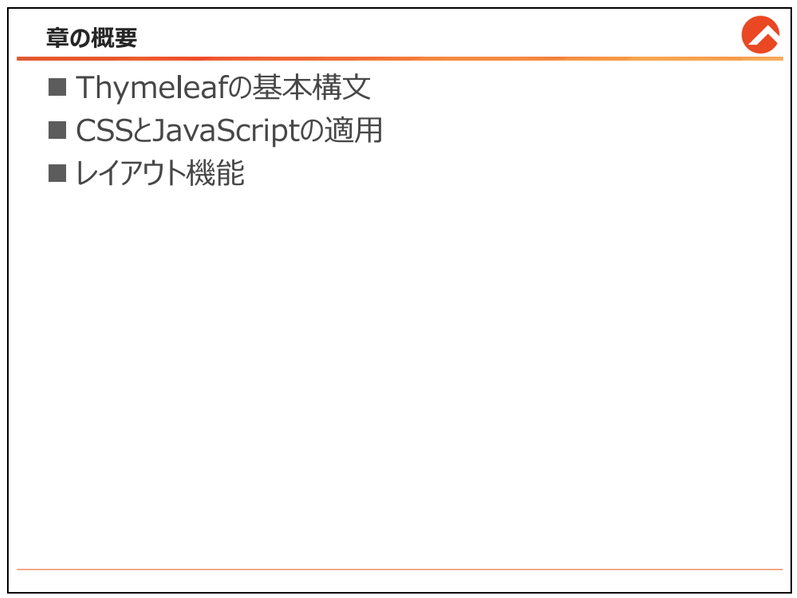
章の概要



今まで、thymeleaf のタグについて、いくつか例文の中でご紹介しました。この章では、改めて thymeleaf の文法をメインの内容にし、CSS と JavaScript を連携する方法についてご説明します。そして、保守性と効率を高めるレイアウト機能についてもご説明します。

第 5 章　Thymeleaf と Web ページデザイン

章の概要



今まで、thymeleaf のタグについて、いくつか例文の中でご紹介しました。この章では、改めて thymeleaf の文法をメインの内容にし、CSS と JavaScript を連携する方法についてご説明します。そして、保守性と効率を高めるレイアウト機能についてもご説明します。

5.1. Thymeleaf の基本構文

Thymeleafの基本構文



- HTMLタグに「th:text」や「th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。
- 以下のコードで、thymeleaf独自の属性・式を利用可能

```
<html xmlns:th="http://www.thymeleaf.org">
```

- th:text
  - **例）<td th:text="${product.id}"></td>**
  - 自動エスケープ機能が内蔵され、
    <script>悪質なコード</script>
    上記のようなデータを無効化できる

第 5 章 Thymeleaf と Web ページデザイン

5.1. Thymeleaf の基本構文

Thymeleafの基本構文



- HTMLタグに「th:text」や「th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。
- 以下のコードで、thymeleaf独自の属性・式を利用可能

```
<html xmlns:th="http://www.thymeleaf.org">
```

- th:text
  - **例）<td th:text="${product.id}"></td>**
  - 自動エスケープ機能が内蔵され、
    <script>悪質なコード</script>
    上記のようなデータを無効化できる

Thymeleaf について

「Thymeleaf」は、JSP と同様に、Java の「テンプレートエンジン」の一種で、HTML5 形式のテンプレートを変換し、Web ブラウザでの表示を可能にするライブラリです。Thymeleaf は、コントローラーからのデータを受け取ることができますので、コンテンツの動的な変更が実現できます。Thymeleaf の主要な記述方式は以下の通りです。

・属性記述：HTML タグに「th:text」や「th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。

・式記述：属性の値を「${}」や「@{}」の形式で表現し、動的なデータの挿入ができる。

Thymeleaf は Spring Framework との統合が容易であり、Spring アプリケーションでの使用に特に適しています。HTML の中に自然にデータを組み込むことができるため、可読性も高いです。さらに、Thymeleaf はセキュリティ面での配慮もなされており、XSS（クロスサイトスクリプティング）対策などのセキュリティ機能を内蔵しています。

Thymeleaf について

「 Thymeleaf」は、JSP と同様に、Java の「 テンプレートエンジン」の一種で、HTML5 形式のテンプレートを変換し、Web ブラウザでの表示を可能にするライブラリです。Thymeleaf は、コントローラーからのデータを受け取ることができますので、コンテンツの動的な変更が実現できます。Thymeleaf の主要な記述方式は以下の通りです。

・属性記述：HTML タグに「 th:text」や「 th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。

・式記述：属性の値を「 ${}」や「 @{}」の形式で表現し、動的なデータの挿入ができる。

Thymeleaf は Spring Framework との統合が容易であり、Spring アプリケーションでの使用に特に適しています。HTML の中に自然にデータを組み込むことができるため、可読性も高いです。さらに、Thymeleaf はセキュリティ面での配慮もなされており、XSS(クロスサイトスクリプティング）対策などのセキュリティ機能を内蔵しています。

第 5 章 Thymeleaf と Web ページデザイン

Thymeleaf を使うための事前準備

HTML ファイル内で、Thymeleaf の属性や式を使うためには、以下のようなコードを記載しておく必要があります。

```
1:<!DOCTYPE html>
2:<html xmlns:th="http://www.thymeleaf.org">
3:
4:<head>
5:<meta charset="UTF-8" />
6:<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```


1:<!DOCTYPE html>
2:<html xmlns:th="http://www.thymeleaf.org">
3:
4:<head>
5:<meta charset="UTF-8" />
6:<meta name="viewport" content="width=device-width, initial-scale=1.0" />

・2 行 <html xmlns:th="http://www.thymeleaf.org">:

Thymeleaf テンプレートで使われる HTML の宣言です。この宣言により、Thymeleaf のルールが HTML ページに追加されます。それによって、Thymeleaf の特有の属性や構文が HTML タグ内で認識され、正しく処理されます。

・6 行 <meta name="viewport" content="width=device-width, initial-scale=1.0" />:

この宣言は、Thymeleaf の構文ではありませんが、Web ページがモバイルデバイスで表示される際のビューポート(表示領域)の設定に使われます。デバイスがどのような画面サイズを持っていても、Web ページはそれに適応するように表示されます。

th:text について

```
<td th:text="${product.id}">[データがあったら、置き換えられる]</td>
<td th:text="${product.name}">[データがあったら、置き換えられる]</td>
<td th:text="${product.price}">[データがあったら、置き換えられる]</td>
```

一番基本となる、th:text は、HTML の要素に動的なテキスト内容を挿入するために使用されます。主に、サーバー側で処理されるデータ(例えば、データベースから取得した情報やユーザ入力)を Web ページに表示する際に用います。

<td></td> タグ内のテキストは、th:text 属性に指定された式の結果によって置き換えられます。

Thymeleaf テンプレートで使われる HTML の宣言です。この宣言により、Thymeleaf のルールが HTML ページに追加されます。それによって、Thymeleaf の特有の属性や構文が HTML タグ内で認識され、正しく処理されます。

この宣言は、Thymeleaf の構文ではありませんが、Web ページがモバイルデバイスで表示される際のビューポート(表示領域)の設定に使われます。デバイスがどのような画面サイズを持っていても、Web ページはそれに適応するように表示されます。

一番基本となる、th:text は、HTML の要素に動的なテキスト内容を挿入するために使用されます。主に、サーバー側で処理されるデータ(例えば、データベースから取得した情報やユーザ入力)を Web ページに表示する際に用います。

<td></td> タグ内のテキストは、th:text 属性に指定された式の結果によって置き換えられます。

168

th:text の特徴の一つは、クロスサイトスクリプティング（XSS）の対策です。例えば、ユーザが「<script>悪質な JavaScript コード</script>」というデータを入力した場合、XSS 対策が施されていないと、この悪意のある JavaScript コードが HTML 内に組み込まれ、不特定多数のユーザに被害を及ぼす可能性があります。

その対策として、th:text を使用すると、入力データは自動的にエスケープ処理されます。これにより、‘<’や‘>’のような文字は HTML タグとしてではなく、ただの文字列として扱われます。結果として、悪意のある JavaScript コードが実行される心配はなくなります。

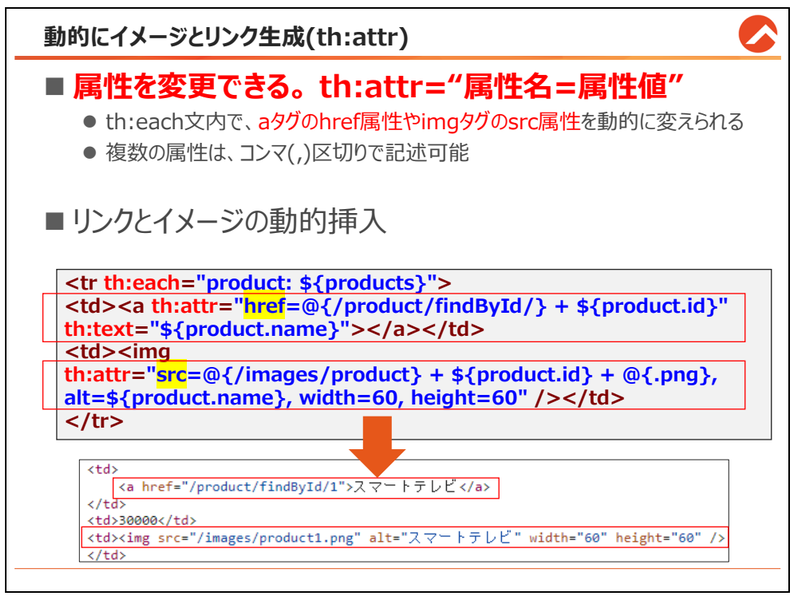
第 5 章 Thymeleaf と Web ページデザイン

th:text の特徴の一つは、クロスサイトスクリプティング(XSS)の対策です。例えば、ユーザが「 <script>悪質な JavaScript コード</script>」というデータを入力した場合、XSS 対策が施されていないと、この悪意のある JavaScript コードが HTML 内に組み込まれ、不特定多数のユーザに被害を及ぼす可能性があります。

その対策として、th:text を使用すると、入力データは自動的にエスケープ処理されます。これにより、‘<’や‘>’のような文字は HTML タグとしてではなく、ただの文字列として扱われます。結果として、悪意のある JavaScript コードが実行される心配はなくなります。

169

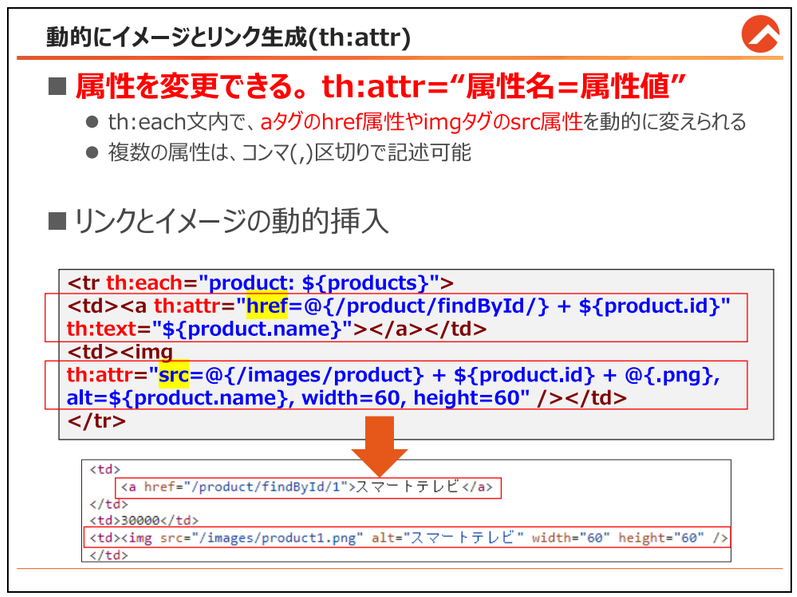
5.1.1. 動的にイメージとリンク生成(th:attr)



Thymeleaf の th:attr 属性を使用することで、HTML 要素の属性を動的に変更することが可能です。データベースから取得したデータに基づいて異なる属性値を設定することができますので、**Form タグ内の id や class 属性を動的で割り当てることができますし、画像の src 属性やリンクの href 属性などを動的に変えることもできます。**

第 5 章 Thymeleaf と Web ページデザイン

5.1.1. 動的にイメージとリンク生成(th:attr)

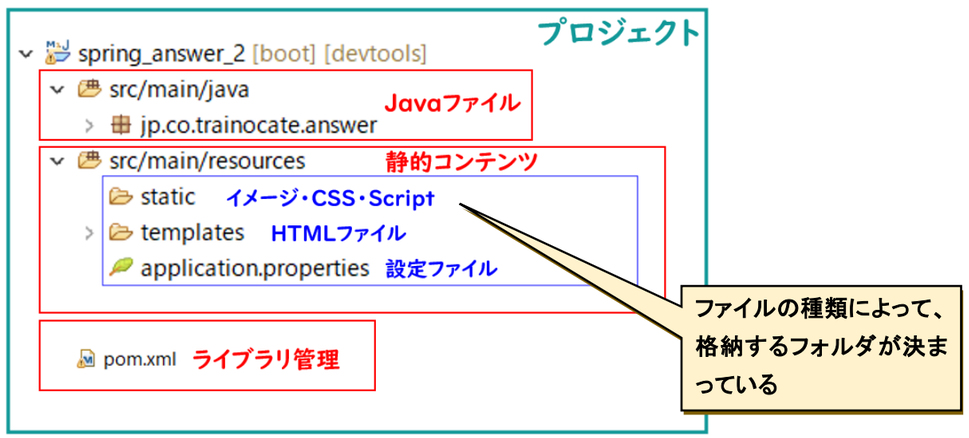


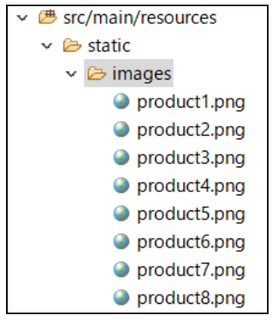
Thymeleaf の th:attr 属性を使用することで、HTML 要素の属性を動的に変更することが可能です。データベースから取得したデータに基づいて異なる属性値を設定することができますので、**Form タグ内の id や class 属性を動的で割り当てることができますし、画像の src 属性やリンクの href 属性などを動的に変えることもできます。**

第 5 章 Thymeleaf と Web ページデザイン

イメージの動的挿入

【振り返り】＜Spring Boot プロジェクトの構成＞





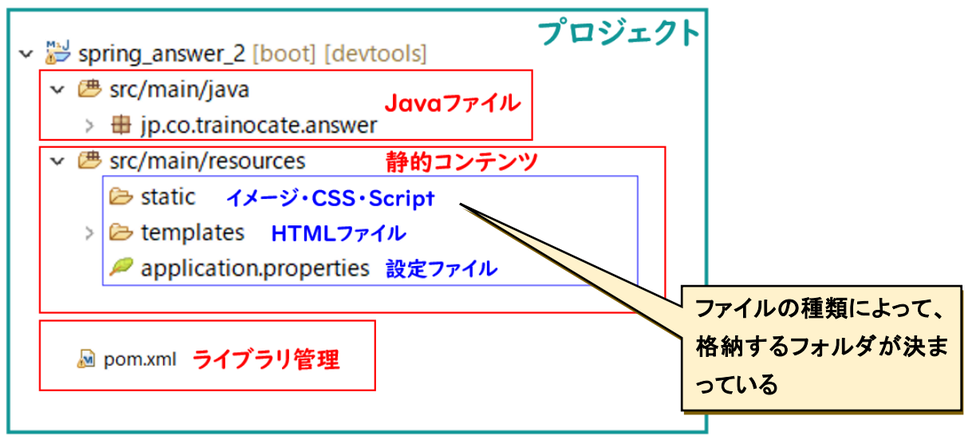
Spring Boot 環境では、イメージのような静的ファイルを src/main/resources/static フォルダに格納する必要があります。今回は、予め images フォルダに、商品ごとのイメージを格納しておきました。product の後ろの番号は、商品番号に合わせています。

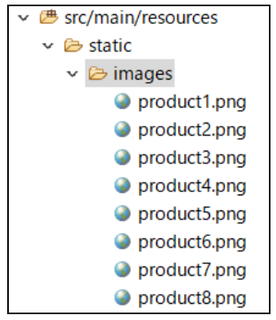
…/resources/templates/product/list.html

```
<table>
<tr>
        <th>商品番号</th>
        <th>商品名</th>
        <th>価格</th>
        <th>イメージ</th>
```


イメージの動的挿入

【 振り返り】＜Spring Boot プロジェクトの構成＞





Spring Boot 環境では、イメージのような静的ファイルを src/main/resources/static フォルダに格納する必要があります。今回は、予め images フォルダに、商品ごとのイメージを格納しておきました。product の後ろの番号は、商品番号に合わせています。

…/resources/templates/product/list.html

```
<table>
<tr>
        <th>商品番号</th>
        <th>商品名</th>
        <th>価格</th>
        <th>イメージ</th>
```

```
</tr>
<tr th:each="product: ${products}">
        <td th:text="${product.id}"></td>
        <td th:text="${product.name}"></td>
        <td th:text="${product.price}"></td>
        <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
                    alt=${product.name}, width=60, height=60" /></td>
</tr>
```

既存の findByAll のサンプルの list.html に、th:attr のコードを追記しました。

th:attr の構文は **th:attr="属性名=属性値"** という形で、属性値を動的に設定できます。通常、イメージを挿入する際は <img src="イメージの URL"> として src 属性に URL を指定しますが、**th:attr を使用すると <img th:attr="src=@{イメージの URL}"> のように書き換えることができます**。

この例では、product.id を用いて商品ごとに異なるイメージ URL を生成しています。th:attr を使用する際に複数の属性を指定したい場合は、コンマ(,)で区切ることで設定できます。alt 属性はイメージの表示が失敗した時の代替テキストとして機能し、ここでは商品名を出力しています。また、width と height 属性を指定することでイメージの大きさを統一しています。

出力結果をみると、以下のように変換されているのがわかります。

```
<tr>

    <td>1</td>
    <td>スマートテレビ</td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" /></td>
</tr>
<tr>

    <td>2</td>
    <td>ゲーミングモニター</td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" /></td>
</tr>
<tr>

    <td>3</td>
    <td>ノートパソコン</td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" /></td>
</tr>
```


第 5 章 Thymeleaf と Web ページデザイン


```
</tr>
<tr th:each="product: ${products}">
        <td th:text="${product.id}"></td>
        <td th:text="${product.name}"></td>
        <td th:text="${product.price}"></td>
        <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
                    alt=${product.name}, width=60, height=60" /></td>
</tr>
```

既存の findByAll のサンプルの list.html に、th:attr のコードを追記しました。

th:attr の構文は **th:attr="属性名=属性値"** という形で、属性値を動的に設定できます。通常、イメージを挿入する際は <img src="イメージの URL"> として src 属性に URL を指定しますが、**th:attr を使用すると <img th:attr="src=@{イメージの URL}"> のように書き換えることができます**。

この例では、product.id を用いて商品ごとに異なるイメージ URL を生成しています。th:attr を使用する際に複数の属性を指定したい場合は、コンマ(,)で区切ることで設定できます。alt 属性はイメージの表示が失敗した時の代替テキストとして機能し、ここでは商品名を出力しています。また、width と height 属性を指定することでイメージの大きさを統一しています。

出力結果をみると、以下のように変換されているのがわかります。

```
<tr>

    <td>1</td>
    <td>スマートテレビ</td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" /></td>
</tr>
<tr>

    <td>2</td>
    <td>ゲーミングモニター</td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" /></td>
</tr>
<tr>

    <td>3</td>
    <td>ノートパソコン</td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" /></td>
</tr>
```

第 5 章 Thymeleaf と Web ページデザイン

動的 URL のリンク挿入

Web システムには、以下のようにデータごとに動的なリンクがかかっていることもよくあります。



商品一覧

商品番号	商品名	価格	イメージ
1	スマートテレビ	30000	
2	ゲーミングモニター	20000	
3	ノートパソコン	50000	
4	2ドア冷蔵庫	50000	
5	コンパクト冷蔵庫	15000	
6	ワインクーラー	30000	
7	全自動洗濯機	25000	
8	乾燥機能付き洗濯機	40000	

この場合、**a タグに同じく th:attr 属性値として href を指定すると、動的リンクを生成**することができます。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
   <td th:text="${product.id}"></td>
   <td>
      <a th:attr="href=@{/product/findById/} + ${product.id}"
      th:text="${product.name}"></a>
   </td>
   <td th:text="${product.price}"></td>
   <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
         alt=${product.name}, width=60, height=60" />
   </td>
</tr>
```


動的 URL のリンク挿入

Web システムには、以下のようにデータごとに動的なリンクがかかっていることもよくあります。



商品一覧

商品番号	商品名	価格	イメージ
1	スマートテレビ	30000	
2	ゲーミングモニター	20000	
3	ノートパソコン	50000	
4	2ドア冷蔵庫	50000	
5	コンパクト冷蔵庫	15000	
6	ワインクーラー	30000	
7	全自動洗濯機	25000	
8	乾燥機能付き洗濯機	40000	

この場合、**a タグに同じく th:attr 属性値として href を指定すると、動的リンクを生成**することができます。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
   <td th:text="${product.id}"></td>
   <td>
      <a th:attr="href=@{/product/findById/} + ${product.id}"
      th:text="${product.name}"></a>
   </td>
   <td th:text="${product.price}"></td>
   <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
         alt=${product.name}, width=60, height=60" />
   </td>
</tr>
```

…/controller/ProductController.java

```
@GetMapping("/findById/{id}")
public String showProductById(@PathVariable int id, Model model) {
        Product product = prodRepo..findById(id).orElse(null);
        model.addAttribute("product", product);
        return "product/detail";
}
```

動的なURLリンクによる呼び出しの場合は、URLからidを受け取る処理が必要となりますので、2章のupdateとdeleteの場合と同様に、@PathVariableアノテーションを付与する必要があります。(その場合、フォームからのid検索は無効になります)

動作確認

http://localhost:8080/product/findAll

正しく指定した場合、以下のようなコードが生成されることを確認できます。

```
<tr>
    <td>1</td>
    <td>
        <a href="/product/findById/1">スマートテレビ</a>
    </td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>2</td>
    <td>
        <a href="/product/findById/2">ゲーミングモニター</a>
    </td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>3</td>
    <td>
        <a href="/product/findById/3">ノートパソコン</a>
    </td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" />
    </td>
</tr>
```


第 5 章 ThymeleafとWebページデザイン

…/controller/ProductController.java

```
@GetMapping("/findById/{id}")
public String showProductById(@PathVariable int id, Model model) {
        Product product = prodRepo..findById(id).orElse(null);
        model.addAttribute("product", product);
        return "product/detail";
}
```

動的なURLリンクによる呼び出しの場合は、URLからidを受け取る処理が必要となりますので、2章のupdateとdeleteの場合と同様に、@PathVariableアノテーションを付与する必要があります。(その場合、フォームからのid検索は無効になります)

動作確認

http://localhost:8080/product/findAll

正しく指定した場合、以下のようなコードが生成されることを確認できます。

```
<tr>
    <td>1</td>
    <td>
        <a href="/product/findById/1">スマートテレビ</a>
    </td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>2</td>
    <td>
        <a href="/product/findById/2">ゲーミングモニター</a>
    </td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>3</td>
    <td>
        <a href="/product/findById/3">ノートパソコン</a>
    </td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" />
    </td>
</tr>
```

174

5.1.2. 条件式による表示制御(th:if)



①th:if と th:unless

Thymeleaf では、th:if や th:unless を活用して、条件に基づいた HTML の表示を制御することができます。これにより、データベースから取得した**データに基づいて異なる表示を行う**ことが可能です。

活用例は多岐にわたりますが、以下にその一例をご紹介します。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
<!--省略-->

<!-- 価格が 30000 より大きい場合、赤色で価格を表示 -->
<td th:if="${product.price > 30000}" th:text="${product.price}"
style="color: red;"></td>

<!-- 価格が 30000 以下の場合、通常のスタイルで価格を表示 -->
<td th:unless="${product.price > 30000}"
th:text="${product.price}"></td>
```


第 5 章 Thymeleaf と Web ページデザイン

5.1.2. 条件式による表示制御(th:if)



①th:if と th:unless

Thymeleaf では、th:if や th:unless を活用して、条件に基づいた HTML の表示を制御することができます。これにより、データベースから取得した**データに基づいて異なる表示を行う**ことが可能です。

活用例は多岐にわたりますが、以下にその一例をご紹介します。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
<!--省略-->

<!-- 価格が 30000 より大きい場合、赤色で価格を表示 -->
<td th:if="${product.price > 30000}" th:text="${product.price}"
style="color: red;"></td>

<!-- 価格が 30000 以下の場合、通常のスタイルで価格を表示 -->
<td th:unless="${product.price > 30000}"
th:text="${product.price}"></td>
```

175

この例では、product.price の値が 30000 を超えるかどうかで、スタイルが適用された価格と、通常スタイルの価格を切り替えて表示しています。**HTML コードは 2 行ですが、条件に合うコードだけが表示されるようになっています。**

②その他の条件式の例

条件式による制御は、活用場面が無限に近いのですが、下記によく使われる例をご紹介します。

⑴ **削除フラグに基づくデータの表示**:

削除フラグが 0 の場合はデータを表示し、1 の場合は表示しない。
(テーブルに deleteFlag の列を追加する必要があります。)

```
<div th:each="product : ${products}" th:if="${product.deleteFlag == 0}"
th:text="${product.name}"></div>
```

⑵ **管理者権限に基づく機能の表示**:

特定のユーザー(例えば、管理者)にのみ特定の機能を表示する。

(テーブルに role の列を追加する必要があります。)

```
<div th:if="${session.user.role == 'ADMIN'}">
    <!-- 管理者専用の機能やリンクをここに表示 -->
</div>
```

⑶ **リストの要素が空かどうかのチェック**:

リストが空の場合に、「情報がありません」というメッセージを表示する。

(#lists.isEmpty(products)は、Thymeleaf で提供する変数式です。
簡単にリストの検索 OK/NG を判定できる構文です。)

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```


第 5 章 Thymeleaf と Web ページデザイン

この例では、product.price の値が 30000 を超えるかどうかで、スタイルが適用された価格と、通常スタイルの価格を切り替えて表示しています。**HTML コードは 2 行ですが、条件に合うコードだけが表示されるようになっています。**

②その他の条件式の例

条件式による制御は、活用場面が無限に近いのですが、下記によく使われる例をご紹介します。

⑴ **削除フラグに基づくデータの表示**:

削除フラグが 0 の場合はデータを表示し、1 の場合は表示しない。
(テーブルに deleteFlag の列を追加する必要があります。)

```
<div th:each="product : ${products}" th:if="${product.deleteFlag == 0}"
th:text="${product.name}"></div>
```

⑵ **管理者権限に基づく機能の表示**:

特定のユーザー(例えば、管理者)にのみ特定の機能を表示する。

(テーブルに role の列を追加する必要があります。)

```
<div th:if="${session.user.role == 'ADMIN'}">
    <!-- 管理者専用の機能やリンクをここに表示 -->
</div>
```

⑶ **リストの要素が空かどうかのチェック**:

リストが空の場合に、「 情報がありません」というメッセージを表示する。

(#lists.isEmpty(products)は、Thymeleaf で提供する変数式です。
簡単にリストの検索 OK/NG を判定できる構文です。)

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```

第 5 章 Thymeleaf と Web ページデザイン

⑷ 日付や時間に基づくコンテンツの表示：

現在の日付や時間に基づいて、特定のメッセージやコンテンツを表示する。

（**#dates.〜〜**は、Thymeleaf で提供する変数式です。
簡単に現在の日付関連のデータを取得できる構文です。）

```
<div th:if="${#dates.dayOfWeek(...) == 2 }">
    今日は月曜特別セールです！
</div>
```

⑷ 日付や時間に基づくコンテンツの表示：

現在の日付や時間に基づいて、特定のメッセージやコンテンツを表示する。

（**#dates.〜〜**は、Thymeleaf で提供する変数式です。
簡単に現在の日付関連のデータを取得できる構文です。）

    今日は月曜特別セールです！

⑸ Radio ボタンのチェックの切り替え：

値が 1 か 2 かによって、ラジオボタンの checked 属性を動的に切り替える。

Radio ボタン以外にも、checkbox でも活用できます。

※if-unless でも書けますが、th:checked 属性を使うと、より簡略に記述できます。

```
<input type="radio" name="gender" th:value="1" th:checked="${gender == 1}" />
<input type="radio" name="gender" th:value="2" th:checked="${gender == 2}" />
```

⑸ Radio ボタンのチェックの切り替え：

値が 1 か 2 かによって、ラジオボタンの checked 属性を動的に切り替える。

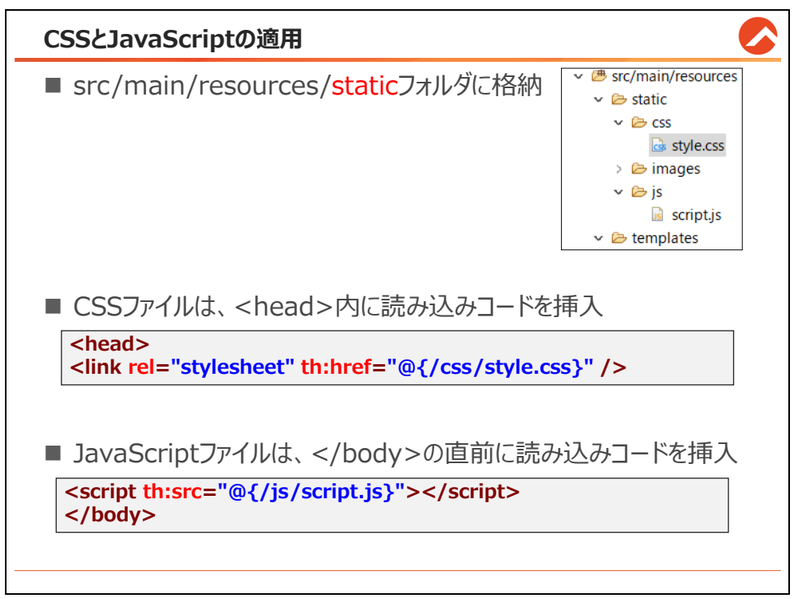
Radio ボタン以外にも、checkbox でも活用できます。

※if-unless でも書けますが、th:checked 属性を使うと、より簡略に記述できます。

<input type="radio" name="gender" th:value="1" th:checked="${gender == 1}" />
<input type="radio" name="gender" th:value="2" th:checked="${gender == 2}" />

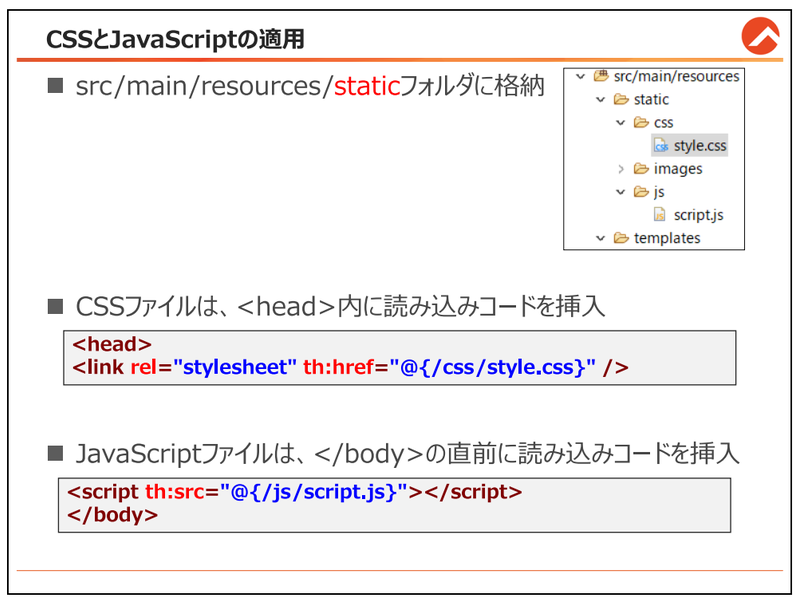
177

5.2. CSS と JavaScript の利用



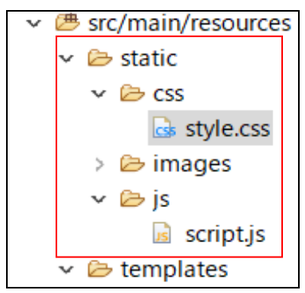
第 5 章 Thymeleaf と Web ページデザイン

5.2. CSS と JavaScript の利用



Thymeleaf 内で CSS ファイルと、JavaScript(JS)ファイルを適用するためには、通常の方法では上手くいかないため、この節でご説明いたします。

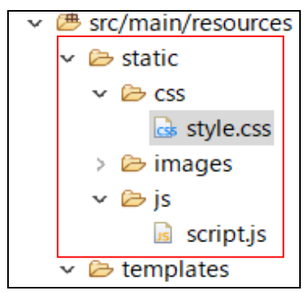
CSS ファイルと JS ファイルの格納場所



css ファイルと js ファイルは、images と同様に **src/main/resources/static の中に格納しないと Spring Boot プロジェクト内で認識されませんので、必ずこの場所に格納してください。**

Thymeleaf 内で CSS ファイルと、JavaScript(JS)ファイルを適用するためには、通常の方法では上手くいかないため、この節でご説明いたします。

CSS ファイルと JS ファイルの格納場所



css ファイルと js ファイルは、images と同様に **src/main/resources/static の中に格納しないと Spring Boot プロジェクト内で認識されませんので、必ずこの場所に格納してください。**

第 5 章 Thymeleaf と Web ページデザイン

CSS ファイルの読み込み

…/resources/templates/product2/list.html

```
<head>
    <link rel="stylesheet" th:href="@{/css/style.css}" />
```

head タグの中に、th:href でファイルパスを記述すれば読み込みできます。/static までのパスは省略されます。

CSS ファイルの読み込み

…/resources/templates/product2/list.html

```
<head>
    <link rel="stylesheet" th:href="@{/css/style.css}" />
```

head タグの中に、th:href でファイルパスを記述すれば読み込みできます。/static までのパスは省略されます。

JS ファイルの読み込み

…/resources/templates/product2/list.html

```
<script th:src="@{/js/script.js}"></script>
</body>
```

body の終了タグの直前の場所に、th:src で js ファイルの場所を記述すると、読み込みできます。

JS ファイルの読み込み

…/resources/templates/product2/list.html

```
<script th:src="@{/js/script.js}"></script>
</body>
```

body の終了タグの直前の場所に、th:src で js ファイルの場所を記述すると、読み込みできます。

サンプルの CSS・JS ファイルのコード

…/resources/static/css/style.css

```
@charset "UTF-8";
body{background-color: black;
     color: white;}
h1{color: yellow;}
```

単純に背景と文字色を反転させるスタイルが記載されています。

…/resources/static/js/script.js

```
document.write("<p>このテキストは JavaScript が生成しました</p>");
```

JavaScript で HTML を生成することもできます。

サンプルの CSS・JS ファイルのコード

…/resources/static/css/style.css

```
@charset "UTF-8";
body{background-color: black;
     color: white;}
h1{color: yellow;}
```

単純に背景と文字色を反転させるスタイルが記載されています。

…/resources/static/js/script.js

```
document.write("<p>このテキストは JavaScript が生成しました</p>");
```

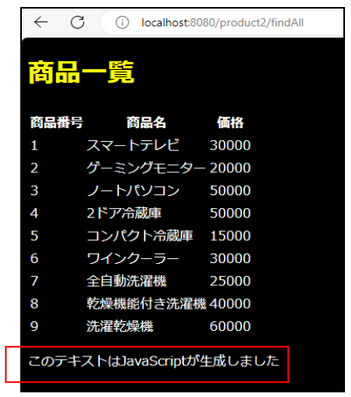
JavaScript で HTML を生成することもできます。

179

第 5 章 ThymeleafとWebページデザイン

動作確認

http://localhost:8080/product2/findAll

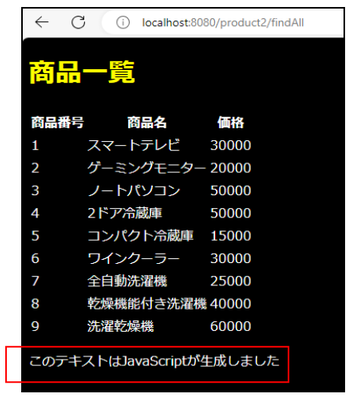


CSS と JavaScript を連携することで、より綺麗で使いやすい画面を作ることができます。

この研修で詳細は割愛しますが、Spring Boot は REST APIも簡単に作成することができますので、Ajax などで非同期処理を混ぜることで、アプリ感覚の SPA(Single Page Application)を作るのも容易です。

動作確認

http://localhost:8080/product2/findAll



CSS と JavaScript を連携することで、より綺麗で使いやすい画面を作ることができます。

この研修で詳細は割愛しますが、Spring Boot は REST APIも簡単に作成することができますので、Ajax などで非同期処理を混ぜることで、アプリ感覚の SPA(Single Page Application)を作るのも容易です。

180

5.3. レイアウト機能

レイアウト機能

■ 共通のデザイン(レイアウト)ファイル

● 以下の属性をレイアウト用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
```

● 共通デザイン定義後、コンテンツが配置される場所を用意

```
<main layout:fragment="main_content">
<!-- 各ページ固有のコンテンツがここに入ります -->
</main>
```

■ コンテンツ用のHTMLファイル

● 以下の属性をコンテンツ用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
layout:decorate="~{ layout.htmlのURL }">
```

● コンテンツをlayout:fragmentのコードで囲むだけ

```
<main layout:fragment="main_content">  ~コンテンツ~  </main>
```

第 5 章 Thymeleaf と Web ページデザイン

5.3. レイアウト機能

レイアウト機能

■ 共通のデザイン(レイアウト)ファイル

● 以下の属性をレイアウト用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
```

● 共通デザイン定義後、コンテンツが配置される場所を用意

```
<main layout:fragment="main_content">
<!-- 各ページ固有のコンテンツがここに入ります -->
</main>
```

■ コンテンツ用のHTMLファイル

● 以下の属性をコンテンツ用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
layout:decorate="~{ layout.htmlのURL }">
```

● コンテンツをlayout:fragmentのコードで囲むだけ

```
<main layout:fragment="main_content">  ~コンテンツ~  </main>
```

共通の要素を適用する課題について

一般的に、Web サイトは画面ごとに共通のデザインテーマを持ち、統一されたヘッダー、サイドバー、フッターを採用しています。しかし、Web サイトは数多くの html ファイルで構成されるものです。全ての html ファイルに、ヘッダー・サイドバー・フッターの要素をコピーすることを考えると、作業量も多いですし、保守性も落ちてしまいます。

Thymeleaf はこの問題を解決するための「レイアウト機能」を提供しています。**一度共通のレイアウトファイルを作成すれば、それを各ページに容易に適用することができる**ため、開発プロセスが大いに効率化されます。

それでは、Thymeleaf でのレイアウトの作成方法について詳しくご説明します。

共通の要素を適用する課題について

一般的に、Web サイトは画面ごとに共通のデザインテーマを持ち、統一されたヘッダー、サイドバー、フッターを採用しています。しかし、Web サイトは数多くの html ファイルで構成されるものです。全ての html ファイルに、ヘッダー・サイドバー・フッターの要素をコピーすることを考えると、作業量も多いですし、保守性も落ちてしまいます。

Thymeleaf はこの問題を解決するための「 レイアウト機能」を提供しています。**一度共通のレイアウトファイルを作成すれば、それを各ページに容易に適用することができる**ため、開発プロセスが大いに効率化されます。

それでは、Thymeleaf でのレイアウトの作成方法について詳しくご説明します。

181

第 5 章 Thymeleaf と Web ページデザイン

目標画面



今回の研修では、メイン画面は画面ごと置き換えできるようにし、赤枠以外の部分を layout.html に記述します。

目標画面



今回の研修では、メイン画面は画面ごと置き換えできるようにし、赤枠以外の部分を layout.html に記述します。

【事前準備】ライブラリを導入

pom.xml

```
<dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

レイアウト機能を使うためには、関連のライブラリが必要です。追加して、maven install をしておきましょう。

【 事前準備】ライブラリを導入

pom.xml

```
<dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

レイアウト機能を使うためには、関連のライブラリが必要です。追加して、maven install をしておきましょう。

182

【作業手順①】レイアウトファイルの作成

…/resources/templates/layout.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org"
 3: xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
 4:
 5:<head>
 6:<meta charset="UTF-8" />
 7:<title>Traino 商品</title>
 8:<link rel="stylesheet" th:href="@{/css/layout.css}" />
 9:</head>
10:<body>
11:<!-- ヘッダー -->
12:<header>
13:<h1>Traino 家電商品</h1>
14:</header>
15:<div class="flex-container">
16:<!-- サイドバー -->
17:<nav>
18:     <ul>
19:          <li><a th:href="@{/product/findAll}">全商品一覧</a></li>
20:          <li><a th:href="@{/product/}">商品検索</a></li>
21:          <li><a th:href="@{/product/insert}">商品登録</a></li>
22:          <li><a th:href="@{/5}">ログアウト</a></li>
23:     </ul>
24:</nav>
25:
26:<!-- メインコンテンツ -->
27:<main layout:fragment="main_content">
28:     <!-- 各ページ固有のコンテンツがここに入ります -->
29:</main>
30:</div>
31:<!-- フッター -->
32:<footer>
33:<p>Copyright © 2023 Trainocate Spring. All rights reserved.</p>
34:</footer>
35:</body>
36:</html>
```


第 5 章 ThymeleafとWebページデザイン

【 作業手順①】レイアウトファイルの作成

…/resources/templates/layout.html


```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org"
 3: xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
 4:
 5:<head>
 6:<meta charset="UTF-8" />
 7:<title>Traino 商品</title>
 8:<link rel="stylesheet" th:href="@{/css/layout.css}" />
 9:</head>
10:<body>
11:<!-- ヘッダー -->
12:<header>
13:<h1>Traino 家電商品</h1>
14:</header>
15:<div class="flex-container">
16:<!-- サイドバー -->
17:<nav>
18:     <ul>
19:          <li><a th:href="@{/product/findAll}">全商品一覧</a></li>
20:          <li><a th:href="@{/product/}">商品検索</a></li>
21:          <li><a th:href="@{/product/insert}">商品登録</a></li>
22:          <li><a th:href="@{/5}">ログアウト</a></li>
23:     </ul>
24:</nav>
25:
26:<!-- メインコンテンツ -->
27:<main layout:fragment="main_content">
28:     <!-- 各ページ固有のコンテンツがここに入ります -->
29:</main>
30:</div>
31:<!-- フッター -->
32:<footer>
33:<p>Copyright © 2023 Trainocate Spring. All rights reserved.</p>
34:</footer>
35:</body>
36:</html>
```

この layout.html には、Web サイトの共通要素であるヘッダー、ナビゲーションバー、フッターが定義されています。注目すべき 3 つの箇所をご説明します。

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

レイアウトファイルを定義するためには、既存の html 属性に、xmlns:layout 属性を追記する必要があります。html 要素を 2 回書いたわけではなく、html 要素に 2 つの属性を追加していることに気をつけてください。

・8 行目　<link rel="stylesheet" th:href="@{/css/layout.css}" />

レイアウト専用の css ファイルを別途作成し、読み込ませました。css ファイルの詳細は、デザインの領域ですので、章末の【参考】の部分をご覧ください。

・27-29 行　<main layout:fragment="main_content">　</main>

この部分のコードは、**中身がありません**。その理由は、ここは**メインコンテンツが配置される場所**だからです。
「layout:fragment="識別名"」の属性をつけると、この部分に別の html ファイル(コンテンツ)の fragment 識別名が一致しているコードがコピーされ、**置き換えられます**。

【作業手順②】コンテンツ用の HTML の修正

…/resources/templates/product/insert.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th=" http://www.thymeleaf.org"
 3: xmlns:layout=" http://www.ultraq.net.nz/thymeleaf/layout"
 4:       layout:decorate=" ~{/layout}" >
 5:
 6:       <!-省略-->
 7:
 8:<body>
 9:<main layout:fragment=" main_content" >
10:
11:<!-既存のコンテンツのソースコード-->
12:
13:</main>
14:</body>
```

修正する HTML ファイルは、商品関連のどの HTML ファイルでも構いません。

第 5 章 Thymeleaf と Web ページデザイン

この layout.html には、Web サイトの共通要素であるヘッダー、ナビゲーションバー、フッターが定義されています。注目すべき 3 つの箇所をご説明します。

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

レイアウトファイルを定義するためには、既存の html 属性に、xmlns:layout 属性を追記する必要があります。html 要素を 2 回書いたわけではなく、html 要素に 2 つの属性を追加していることに気をつけてください。

・8 行目　<link rel="stylesheet" th:href="@{/css/layout.css}" />

レイアウト専用の css ファイルを別途作成し、読み込ませました。css ファイルの詳細は、デザインの領域ですので、章末の【 参考】の部分をご覧ください。

・27-29 行　<main layout:fragment="main_content">　</main>

この部分のコードは、**中身がありません**。その理由は、ここは**メインコンテンツが配置される場所**だからです。
「 layout:fragment="識別名"」の属性をつけると、この部分に別の html ファイル(コンテンツ)の fragment 識別名が一致しているコードがコピーされ、**置き換えられます**。

【 作業手順②】コンテンツ用の HTML の修正

…/resources/templates/product/insert.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th=" http://www.thymeleaf.org"
 3: xmlns:layout=" http://www.ultraq.net.nz/thymeleaf/layout"
 4:       layout:decorate=" ~{/layout}" >
 5:
 6:       <!-省略-->
 7:
 8:<body>
 9:<main layout:fragment=" main_content" >
10:
11:<!-既存のコンテンツのソースコード-->
12:
13:</main>
14:</body>
```

修正する HTML ファイルは、商品関連のどの HTML ファイルでも構いません。

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"

レイアウトにコピーするコンテンツ側の HTML にも、同じくレイアウト関連のルールを宣言しておきます。

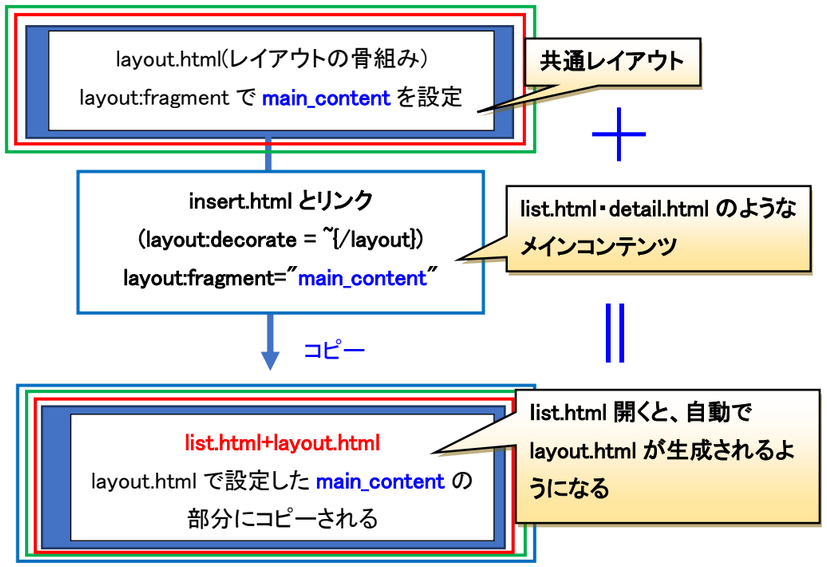
・4 行目　layout:decorate="~{/layout}">

<html>の属性の中に layout:decorate 属性も追記します。この部分の属性値は、**layout.html の場所を URL で指定**すれば OK です。

例えば、layout が product フォルダ内にあるなら、"~{/product/layout}" のように記述すれば OK です。URL を間違えると、レイアウト機能が使えませんので、ご注意ください。

・9-13 行目　<main layout:fragment="main_content">

既存の<body>内に書いていたソースコードを、全てこの要素で囲みます。先ほどの **layout.html の fragment 識別名が"main_content"でしたので、同じ値を入れます**。そうすることで、**9-13 行内の全ての HTML コードが、layout.html の 28 行にコピーされ、レイアウト+コンテンツの新しい HTML が生成されます。insert.html へ return して一覧画面を開くと、自動で layout.html の共通部分まで生成された画面が表示されます。**



今回は、insert.html のみ修正しましたが、レイアウトを適用させたい全ての html ファイルを、同じように修正すれば、以下のようにシステム全体に同じデザインが適用されます。

第 5 章 Thymeleaf と Web ページデザイン

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"

レイアウトにコピーするコンテンツ側の HTML にも、同じくレイアウト関連のルールを宣言しておきます。

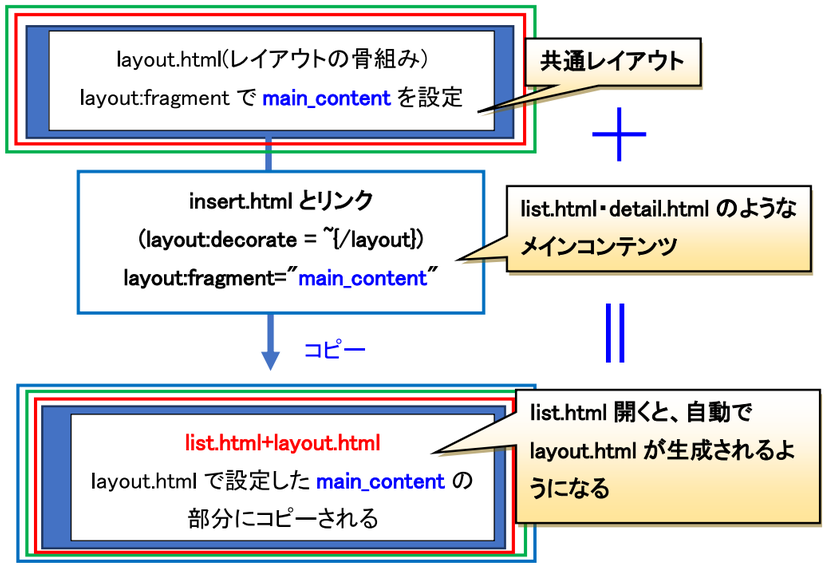
・4 行目　layout:decorate="~{/layout}">

<html>の属性の中に layout:decorate 属性も追記します。この部分の属性値は、**layout.html の場所を URL で指定**すれば OK です。

例えば、layout が product フォルダ内にあるなら、"~{/product/layout}" のように記述すれば OK です。URL を間違えると、レイアウト機能が使えませんので、ご注意ください。

・9-13 行目　<main layout:fragment="main_content">

既存の<body>内に書いていたソースコードを、全てこの要素で囲みます。先ほどの **layout.html の fragment 識別名が"main_content"でしたので、同じ値を入れます**。そうすることで、**9-13 行内の全ての HTML コードが、layout.html の 28 行にコピーされ、レイアウト+コンテンツの新しい HTML が生成されます。insert.html へ return して一覧画面を開くと、自動で layout.html の共通部分まで生成された画面が表示されます。**

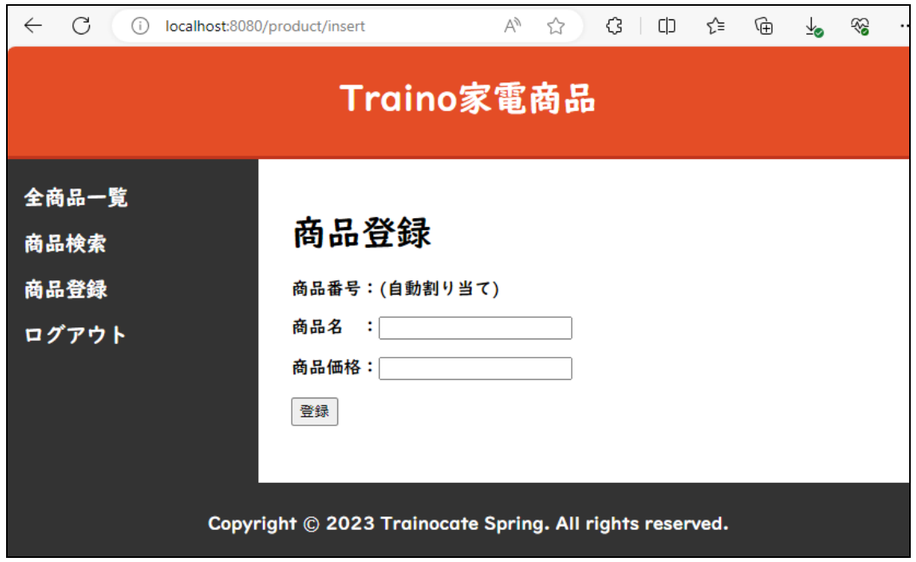


今回は、insert.html のみ修正しましたが、レイアウトを適用させたい全ての html ファイルを、同じように修正すれば、以下のようにシステム全体に同じデザインが適用されます。

第 5 章 ThymeleafとWebページデザイン









186

【参考】layout.css のソースコード

…/resources/static/css/layout.css

```
/* 全体のベーススタイリング */
html, body {
    height: 100%;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    font-family: UD デジタル 教科書体 NP-B, sans-serif;
}

/* ヘッダーのスタイリング */
header {
        background-color: #E44D26;
    color: white;
    padding: 10px 0;
    text-align: center;
    border-bottom: 3px solid #C1351D;
}

/* サイドバーのスタイリング */
nav {
    background-color: #333; /* 黒 */
    color: white;
    padding: 15px;
    width: 200px;
    flex-shrink: 0;
}

nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

nav ul li a {
    color: white;
```


第 5 章 Thymeleaf と Web ページデザイン

【 参考】layout.css のソースコード

…/resources/static/css/layout.css


```
/* 全体のベーススタイリング */
html, body {
    height: 100%;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    font-family: UD デジタル 教科書体 NP-B, sans-serif;
}

/* ヘッダーのスタイリング */
header {
        background-color: #E44D26;
    color: white;
    padding: 10px 0;
    text-align: center;
    border-bottom: 3px solid #C1351D;
}

/* サイドバーのスタイリング */
nav {
    background-color: #333; /* 黒 */
    color: white;
    padding: 15px;
    width: 200px;
    flex-shrink: 0;
}

nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

nav ul li a {
    color: white;
```

```
        text-decoration: none;
        padding: 10px 0;
        display: block;
        font-size: 1.2em; /* テキストサイズを大きくする */
}

nav ul li a:hover {
        background-color: #555;
}

/* メインコンテンツのスタイリング */
main {
        background-color: #fff; /* 白色の背景 */
        padding: 30px;
        overflow-y: auto; /* 必要に応じてスクロールバーを表示 */
        flex-grow: 1;
}

/* フッターのスタイリング */
footer {
        background-color: #333; /* 黒 */
        color: white;
        text-align: center;
        padding: 10px 0;
        width: 100%;
}

/* フレックスボックスのレイアウト */
.flex-container {
        display: flex;
        flex: 1;
        flex-direction: row;
}

@media (max-width: 768px) {
        .flex-container {
                flex-direction: column;
        }
```


第 5 章 ThymeleafとWebページデザイン


```
        text-decoration: none;
        padding: 10px 0;
        display: block;
        font-size: 1.2em; /* テキストサイズを大きくする */
}

nav ul li a:hover {
        background-color: #555;
}

/* メインコンテンツのスタイリング */
main {
        background-color: #fff; /* 白色の背景 */
        padding: 30px;
        overflow-y: auto; /* 必要に応じてスクロールバーを表示 */
        flex-grow: 1;
}

/* フッターのスタイリング */
footer {
        background-color: #333; /* 黒 */
        color: white;
        text-align: center;
        padding: 10px 0;
        width: 100%;
}

/* フレックスボックスのレイアウト */
.flex-container {
        display: flex;
        flex: 1;
        flex-direction: row;
}

@media (max-width: 768px) {
        .flex-container {
                flex-direction: column;
        }
```

```
        nav {
            width: 100%;
        }
}

/* フッターが常に画面の最下部に表示されるようにする */
.flex-container {
        min-height: 0; /* フレックスコンテナの min-height をリセット */
}

body > footer {
        flex-shrink: 0;
}
```

第 5 章 ThymeleafとWebページデザイン

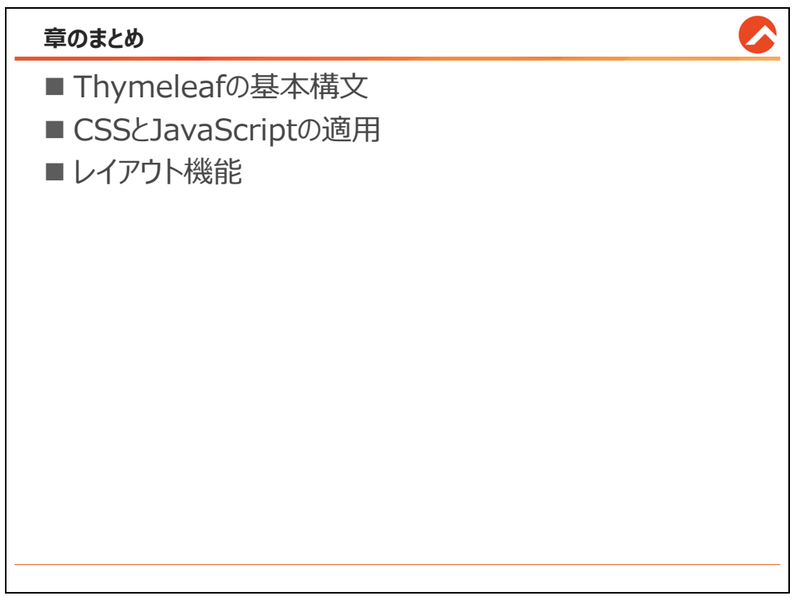
```
        nav {
            width: 100%;
        }
}

/* フッターが常に画面の最下部に表示されるようにする */
.flex-container {
        min-height: 0; /* フレックスコンテナの min-height をリセット */
}

body > footer {
        flex-shrink: 0;
}
```


189

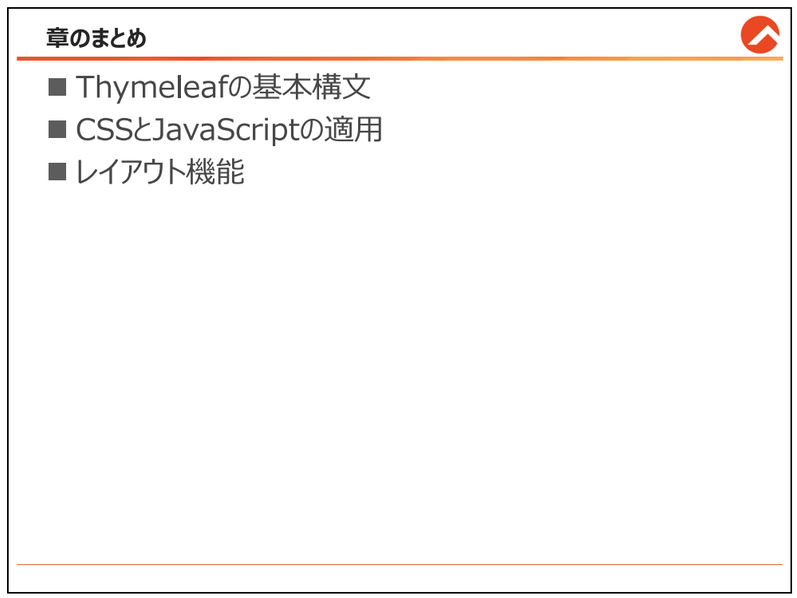
章のまとめ



この章では、Spring Boot でシステムを実装する際に、追加で知っておくべき機能についてご説明しました。本書で全て紹介できなかった Spring の便利な機能もまだまだありますが、一番基本となる内容は網羅しましたので、これから皆様の Spring 開発知識の土台になると確信します。是非マスターしてください。

第 5 章　Thymeleaf と Web ページデザイン

章のまとめ



この章では、Spring Boot でシステムを実装する際に、追加で知っておくべき機能についてご説明しました。本書で全て紹介できなかった Spring の便利な機能もまだまだありますが、一番基本となる内容は網羅しましたので、これから皆様の Spring 開発知識の土台になると確信します。是非マスターしてください。

第 5 章 ThymeleafとWebページデザイン

191